PROJETO INTEGRADOR · FIAP · TECNÓLOGO EM INTELIGÊNCIA ARTIFICIAL

Rede Alimenta IA
Perguntas e respostas sobre a estrutura

Guia de estudo e ensaio da defesa: como o projeto é organizado, por que foi feito assim e o que responder à banca.

Equipe	Davi Oliveira da Silva (RM 569108) João Pedro Morangoni (RM 570073) João Vitor Xavier de Carvalho (RM 570633)
Orientação	Prof. M.Sc Oerton Fernandes · 2026
URL pública	https://alimenta-ia.onrender.com
Repositório	https://github.com/joaovitorxc/Alimenta-IA
Versão	15/10/2026

As respostas descrevem o que o projeto faz hoje, conferido no código, nos testes e em execuções reais. Onde existe uma limitação ou uma proposta ainda não implementada, isso está dito de forma explícita: na defesa, assumir limites com clareza vale mais do que prometer o que não existe.

1. Visão geral

O problema, a solução e como ela se encaixa no enunciado.

1. **Qual problema o projeto resolve?**

   O Brasil descarta toneladas de comida enquanto milhões passam fome. A Rede Alimenta IA conecta quem tem alimento sobrando (padarias, mercados, restaurantes, produtores) a ONGs que precisam receber, antes de o alimento perder a validade. A IA ajuda a decidir *qual ONG* deve receber *qual lote*, priorizando o que corre mais risco de ser descartado.

2. **Quem usa a plataforma e o que cada pessoa faz?**

   Há três papéis:

   - **Doador** (qualquer pessoa, sem conta): envia uma oferta em `/doar` e recebe um **código de acompanhamento**; com ele, em `/acompanhar`, vê em que etapa está a doação. Não vê nenhum dado do sistema.
   - **Gestor (admin)**: entra em `/admin` com **e-mail e senha de uma conta individual**. Aprova ofertas, pede a recomendação de ONGs, cria matches, avisa a ONG (e-mail ou WhatsApp), gerencia o link e os **acessos** de cada ONG, vê a **auditoria** e os indicadores.
   - **ONG**: entra em `/ong` com a **conta de cada pessoa** (e-mail e senha). No primeiro acesso usa o link pessoal que o gestor enviou, que funciona como **convite** para criar a conta. Vê só as doações indicadas para ela, aceita ou recusa e, quando o alimento chega, **confirma o recebimento**. O contato do doador só aparece depois do aceite.

3. **Qual é o fluxo completo de uma doação?**

   1. O doador envia a oferta; ela é validada no servidor e fica numa **fila própria** (`ofertas_publicas`). **Ainda não existe doador nem doação.**
   2. O gestor **aprova** (cria o doador, ou reaproveita o do mesmo e-mail, e a doação *disponível*, numa única transação) ou **recusa** (os dados pessoais da oferta são apagados).
   3. O gestor usa **Recomendar ONG** (IA) e cria o **match**; a doação passa sozinha a *reservada*, e o botão **Avisar ONG** abre e-mail ou WhatsApp com o link.
   4. A ONG **aceita** ou **recusa**; recusar libera a doação de volta a *disponível*.
   5. Quando o alimento chega, a **ONG confirma o recebimento** (ou o gestor clica em **Concluir entrega**): o match vira *concluído*, a doação vira *concluída* e os kg entram nos indicadores, com o registro de quem confirmou.

   Em todas as etapas o doador pode conferir a situação na página de acompanhamento.

4. **Como o projeto cobre os entregáveis do enunciado?**

Entregável	Onde está
Dataset estruturado	`ml/data/lotes_simulados.csv` (3.000 lotes simulados, seed 42)
Modelo funcional local	`ml/train.py`, `ml/predict.py`, `ml/models/` e a inferência na API (`rede-alimenta-back/ia/`)
Matriz STRIDE	`documentacao/04-matriz-stride.md` (28 ameaças com controle e teste)
Dockerfile + Compose	`Dockerfile`, `compose.yaml`
CI/CD (GitHub Actions)	`.github/workflows/devsecops.yml`
SAST (Bandit e Trivy)	Etapas do workflow, com relatórios como artefatos; resultados em `documentacao/05-devsecops-e-deploy.md`
URL pública	https://alimenta-ia.onrender.com (Render)
Banco em nuvem	Supabase (PostgreSQL), esquema em `database/schema.sql` + migrações
README e pitch	`README.md` e `documentacao/08-indicadores-e-pitch.md`

5. **Por que não usaram FastAPI e Streamlit, como sugere o enunciado?**
   Foi uma decisão consciente, registrada na documentação. O servidor é Python puro (`http.server`), o que deixa a imagem Docker pequena (Alpine, apenas 5 pacotes Python), reduz a superfície de ataque e permite uma CSP estrita, sem scripts inline. O frontend é HTML/CSS/JS na mesma origem porque há três perfis de usuário que um painel Streamlit atenderia mal. Mantivemos o que o enunciado exige de fato: API REST, documentação Swagger/OpenAPI, painel interativo e IA em Python.

   > **Ponto fraco a assumir:** o que o FastAPI daria pronto (validação, roteamento, OpenAPI) foi escrito à mão. Se a banca exigir FastAPI, a camada de serviços e repositórios pode ser reaproveitada; só a camada HTTP mudaria.

2. Estrutura do código

Pastas, camadas e responsabilidades.

6. Como o repositório está organizado?

```
Alimenta-IA/
├─ frontend/             páginas (início, /admin, /doar, /acompanhar, /ong, /privacidade), JS e
CSS
├─ rede-alimenta-back/   API e regras de negócio
│  ├─ server.py          servidor HTTP, rotas, limites, cabeçalhos e migração ao iniciar
│  ├─ security.py        senhas (PBKDF2), sessões, token, link da ONG, rate limit
│  ├─ services/          validação e regras (doação, ONG, match, oferta, geo, IA, indicadores…)
│  ├─ repositories/      SQL parametrizado (uma classe de acesso por assunto)
│  ├─ database/          conexão e schema.sql
│  ├─ migracoes/         migrações SQL aditivas (001 a 007)
│  ├─ ia/                modelo em JSON e inferência em Python puro
│  ├─ tests/             212 testes automatizados (não usam o banco)
│  ├─ migrar.py          aplica schema + migrações
│  └─ demo_seed.py       dados de demonstração (só em banco local)
├─ ml/                   dataset, treino, predição e relatórios (scikit-learn)
├─ ferramentas/          gerar_seguranca.py (roda Bandit, pip-audit, Gitleaks, Trivy e testes)
├─ documentacao/         requisitos, arquitetura, IA, STRIDE, DevSecOps, LGPD…
├─ .github/workflows/    pipeline DevSecOps
├─ Dockerfile · compose.yaml · render.yaml
```

7. Qual é a arquitetura em camadas?

Navegador → **frontend** (HTML/CSS/JS) → **server.py** (autenticação, limites, cabeçalhos, rotas) → **services** (validação e regras) → **repositories** (SQL parametrizado) → **connection.py** (psycopg) → **PostgreSQL/Supabase**. A IA fica ao lado dos serviços (`ia/`). Cada camada só fala com a seguinte, o que facilita testar e trocar partes.

8. Por que separar serviços e repositórios?

Os **repositórios** só executam SQL (sempre com parâmetros, nunca concatenando texto). Os **serviços** decidem o que é válido e o que é permitido: faixas de quantidade, validade coerente com a categoria, doador existente, transições de status etc. Assim, as regras ficam em um lugar só, e os testes (sem banco) conseguem simular os repositórios e exercitar as regras.

9. Como é o frontend e por que sem framework?

HTML, CSS e JavaScript simples, servidos pelo próprio backend. Não há build nem Node. A página do gestor é uma aplicação de página única que consome a API REST e se atualiza sozinha a cada 30 s. Isso reduz dependências (menos risco de supply chain) e permite a CSP restritiva: nenhum script ou estilo inline é executado, e um **teste automatizado** garante isso (um erro desse tipo só apareceria no navegador). Os gráficos e o mapa são SVG gerado em JavaScript, sem biblioteca. O layout é responsivo (testado em 1366 px e 390 px).

10. A API é documentada? Como?

Sim. `/docs` abre o Swagger UI (versão 5.33.1, **vendorizado** no repositório, sem CDN, com hashes SHA-256 conferidos no pipeline) e `/openapi.json` traz o contrato, inclusive as rotas públicas e do portal da ONG. O botão Authorize recebe o token do gestor. Atenção: o *Try it out* executa operações reais no banco.

11. Quais são as principais rotas?

Grupo	Rotas	Acesso
CRUD	`/api/doadores`, `/ongs`, `/doacoes`, `/matches` (GET, POST, PUT, DELETE)	Token do gestor
Fluxo	`/api/ofertas` (+ aprovar/recusar); `/api/doacoes/{id}/aprovar` (fila antiga); `/api/ongs/{id}/link`; `/api/ong-solicitacoes`	Token do gestor
IA	`POST /api/ia/risco`; `GET /api/doacoes/{id}/recomendacoes`; `GET /api/indicadores`	Token do gestor
Públicas	`POST /api/publico/doacoes`; `/api/publico/acompanhar`; `/api/publico/ong-acesso`	Sem login, com limites
Portal da ONG	`/api/ong-portal/matches` (+ aceitar, recusar, receber) e `/conta`	Conta da ONG ou link com código
Conta	`POST /api/login`; `/api/eu`, `/api/eu/senha`, `/api/logout`; `/api/usuarios` e `/api/auditoria` (gestor)	E-mail e senha

3. Banco de dados

Supabase (PostgreSQL), esquema versionado e migrações.

12. Quais são as tabelas e como se relacionam?

Quatro tabelas centrais: **doadores**, **ongs**, **doacoes** e **matches**. Um doador tem várias doações; uma doação pode ter vários matches (tentativas com ONGs diferentes); uma ONG tem vários matches. Tabelas de apoio: **ofertas_publicas** (fila de ofertas até a decisão do gestor), **match_confirmacoes** (quem confirmou cada entrega), **ong_links_acesso** (versão e situação do link de cada ONG), **ong_solicitacoes_acesso** (pedidos de acesso de ONGs), **usuarios** e **usuario_sessoes** (contas e sessões, só com hashes) e **auditoria_acoes** (quem fez o quê). O e-mail é único em doadores e ONGs.

13. Como o banco evita reservar a mesma doação duas vezes?

Em duas camadas. No código, criar um match faz, em **uma única transação**, `UPDATE doacoes SET status='reservada' WHERE status='disponivel'`: se duas pessoas tentarem ao mesmo tempo, só uma altera a linha. No banco, dois índices únicos parciais em `matches` impedem mais de um match *aceito/concluído* por doação e a repetição do par (doação, ONG) pendente.

14. Como o esquema é criado e atualizado?

`database/schema.sql` cria as quatro tabelas centrais e `migracoes/001…007` acrescentam as demais. Tudo é **idempotente e aditivo** (`CREATE … IF NOT EXISTS`, `ADD COLUMN IF NOT EXISTS`), e um teste reprova qualquer `DROP`, `DELETE`, `TRUNCATE` ou `UPDATE` nesses arquivos. **O servidor aplica tudo sozinho ao iniciar** (`AUTO_MIGRAR=1`, o padrão), então depois de cada deploy não é preciso lembrar de rodar nada; `python migrar.py` faz o mesmo à mão. Se o banco estiver fora do ar, o servidor sobe mesmo assim e registra o erro.

15. E se o sistema subir antes de o banco ter as tabelas novas?

Continua funcionando, em modo reduzido, e avisa: a oferta pública cai no fluxo antigo (cria o doador e uma doação pendente, sem código de acompanhamento); concluir uma entrega funciona sem o registro de quem confirmou (um *savepoint* isola essa gravação); os pedidos de acesso de ONGs avisam que estão indisponíveis; e os indicadores usam só o que existe. Esses caminhos foram testados com um banco criado só até a migração 004.

16. O que acontece ao excluir um doador ou uma ONG?

No banco, as chaves estrangeiras têm `ON DELETE CASCADE`, que apagaria o histórico. Por isso o servidor **recusa a exclusão** enquanto houver doações ou matches vinculados, com mensagem clara, e o painel pede confirmação com o nome do registro. É o risco T07 da matriz STRIDE.

17. Onde fica o CNPJ da ONG?

Na coluna `ongs.cnpj`, **opcional e único** quando informado (índice único parcial), guardado só com dígitos. O servidor confere os dois dígitos verificadores. Não é exigido porque ONGs pequenas e coletivos informais muitas vezes não têm CNPJ. O painel avisa "Sem CNPJ informado" para o gestor conferir de outro jeito.

18. O banco do Supabase tem tabelas que o código não usa?

Sim: ele guarda também tabelas de contas de ONG e doador, sessões, notificações e auditoria, de outra frente de desenvolvimento da equipe. Este código **não as usa** e não as altera. O `schema.sql` descreve apenas o que o projeto precisa para subir do zero.

4. Regras de negócio e validações

Tudo é validado no servidor, mesmo que o formulário seja contornado.

19. Que validações existem nos cadastros?

- **Doação:** 0,01 a 10.000 kg; validade real, não vencida e coerente com a categoria (hortifrúti até 60 dias, padaria 90, prontos 180, laticínios e carnes 365, não perecível 1.095); sem doação idêntica duplicada.
- **Doador e ONG:** nome, e-mail em minúsculas, telefone brasileiro, região, coordenadas dentro do Brasil, capacidade de 0,01 a 1.000.000 kg; textos sem caracteres de controle ou invisíveis.
- **Match:** doação disponível e não vencida, ONG com capacidade para o volume, sem match ativo repetido, score entre 0 e 1.
- JSON com `NaN`/`Infinity`, tipos trocados e ids fora do intervalo são rejeitados com erro 400.

20. Como o sistema sabe que um alimento é mesmo comida?

Em camadas, da mais barata à mais cara, e **negando por padrão**:

1. **Bloqueio local** de animais, objetos, produtos de limpeza, drogas etc. (a IA não consegue sobrepor).
2. **Lista local** de alimentos conhecidos (ignora acento, plural e quantidade) e confere a categoria.
3. **IA gratuita** para o restante: Groq (principal) e Gemini (reserva automática se um falhar).
4. **Falha segura:** se nada confirmar, o cadastro é recusado (HTTP 503, "tente novamente").

Só o **nome do alimento** é enviado à IA, nunca dados de pessoas, e há limite de 30 consultas por minuto.

21. **Por que a oferta fica numa tabela própria e não cria o doador na hora?**
    Porque qualquer visitante pode enviar uma oferta, e antes ela criava um "doador" no cadastro mesmo que ninguém a aprovasse (a lista de doadores se enchia de gente aleatória). Agora a oferta fica em `ofertas_publicas` com os dados de quem enviou; **só a aprovação cria o doador e a doação**, numa transação. Se for **recusada**, os dados pessoais (nome, e-mail, telefone, região) são apagados na hora e sobram só o alimento e a data, para auditoria. É minimização de dados (LGPD) aplicada ao desenho.

22. **Como o doador acompanha a oferta sem ter conta?**
    Ao enviar, ele recebe um **código aleatório de 128 bits** (aparece uma vez, com botão Copiar e link). Em `/acompanhar` a página mostra quatro etapas: oferta recebida, aprovada, ONG indicada e entregue, com um texto claro em cada uma ("estamos analisando", "uma ONG aceitou", "entregue, obrigado"). O banco guarda só o **hash** do código; o código viaja no corpo de um POST e no fragmento `#` do link (fora de logs e do Referer); a resposta não traz dados pessoais, só etapa, alimento, quantidade e, depois do aceite, o nome da ONG; 20 códigos inválidos por IP em 10 minutos bloqueiam novas tentativas. A página se atualiza sozinha a cada 30 s.

23. **O pedido de acesso da ONG dá acesso automático?**
    Não, de propósito. O pedido (`/ong`) só entra em uma fila no painel do gestor, com nome, e-mail, região, CNPJ e capacidade (opcionais) e mensagem. O gestor confere se a organização é real, cadastra a ONG (o formulário já vem preenchido), gera o link e o envia pelo botão de e-mail ou WhatsApp. A resposta ao pedido é sempre a mesma, para não revelar quais e-mails são de ONGs cadastradas, e descartar um pedido apaga os dados dele.

24. **E se for uma ONG pequena, sem CNPJ?**
    Pode participar. O CNPJ é opcional e não há tamanho mínimo: basta informar a capacidade de recebimento, que pode ser pequena. Quem decide é o gestor, com base no pedido e, se quiser, em contato direto. Se houver CNPJ, ele confere na Receita.

25. **Por que a ONG confirma o recebimento? Isso muda os indicadores?**
    Porque "kg salvos" só conta de verdade quando quem recebeu confirma. Antes, só o gestor concluía a entrega, sem confirmação de quem recebeu. Agora a ONG clica em "Confirmar recebimento" no portal (só nos matches **dela** que ela **aceitou**); o gestor continua podendo concluir. Cada conclusão grava a **origem** (`ong` ou `gestor`) em `match_confirmacoes`, e o dashboard mostra a parcela confirmada pela ONG (cerca de 80% nos dados de demonstração). Continua sem comprovante (foto, assinatura): é a próxima melhoria.

5. Inteligência artificial

Dataset, modelo, métricas e recomendação.

26. **O que a IA prevê?**
    O **risco de descarte** de um lote (baixo, médio ou alto), a partir de: descrição do alimento (texto), categoria, quantidade em kg, dias até a validade, distância até a ONG e capacidade disponível da ONG. O risco alimenta a **prioridade sugerida** e o **score** da recomendação de ONGs.

27. De onde vêm os dados de treino?

De um **dataset simulado** por regras com ruído, gerado por `ml/generate_dataset.py` (3.000 lotes, seed 42, reproduzível). Está identificado como simulado em todos os lugares, inclusive no painel. Não há dados pessoais nele. Não existe conjunto público de doações por lote com a granularidade que o modelo precisa, então **nenhum dado aberto entra no treino**; dados abertos (IBGE, ONU Meio Ambiente, Embrapa) foram usados só para dimensionar o problema, com a fonte e a ressalva registradas em `documentacao/03-dados-e-ia.md`.

> **Limitação a assumir:** como o rótulo vem de uma regra simulada, o modelo aprende a reproduzir essa regra; as métricas demonstram o fluxo, não eficácia em cenário real. Para uso real, é preciso coletar doações e descartes verdadeiros e reavaliar por região e categoria.

28. Qual modelo foi usado e por quê?

Regressão logística com `class_weight="balanced"`, num pipeline scikit-learn com padronização das variáveis numéricas, one-hot da categoria e TF-IDF (1 e 2 gramas) do texto. Foi comparada a um baseline (classe majoritária), à regressão logística simples e a um Random Forest. A escolha usou o **F1 macro na validação**. As regressões logísticas foram as únicas candidatas porque podem ser exportadas para JSON; isso permite inferir em Python puro, sem scikit-learn e sem `pickle` dentro da API.

29. Quais foram os resultados?

Métrica (teste, 450 lotes)	Valor
Acurácia	0,840 (baseline majoritária: 0,700)
F1 macro	0,762
Recall do alto risco	0,892 (precisão 0,949)
Classe "médio" (a mais difícil)	recall 0,622 · precisão 0,535
Latência	~0,013 ms por lote

Divisão estratificada 70/15/15 (treino/validação/teste); o teste foi usado uma única vez, ao final.

30. Por que a acurácia é só 84% e isso é bom?

Porque o baseline que sempre responde "alto risco" já acerta 70%, já que as classes são desbalanceadas. A comparação justa é **84% contra 70%**, e o que mais importa neste problema é o **recall do alto risco (89%)**: não deixar passar lote que vai estragar. A classe "médio" é a mais difícil e vamos dizer isso abertamente.

31. Como funciona a recomendação de ONGs?

Para uma doação *disponível*, o sistema **filtra** as ONGs com capacidade suficiente, **calcula o risco de descarte** para cada uma e **ordena pelo score** (empate: a ONG mais perto), devolvendo até 5 ONGs com o motivo. O score é `1 − (0,5·P(médio) + P(alto))`. A **distância** é a linha reta (haversine) calculada, em ordem de preferência: (1) pelas **coordenadas** do cadastro; (2) pelo **centro da cidade** citada na região (tabela de 107 cidades; "Zona Sul, São Paulo" vira São Paulo); (3) por uma **estimativa fixa** (5 km na mesma região, 15 km em outra). A tela mostra a origem de cada número. A resposta não traz e-mail nem telefone, e há sempre **confirmação humana**: o gestor escolhe e cria o match.

> **Limitação:** linha reta não é rota de coleta, e não há tempo de deslocamento. Rotas são a melhoria prevista.

32. Encontraram algum problema no modelo ao testar? Como resolveram?

Sim, e é um bom exemplo de validação. Ao testar a recomendação com ONGs de portes diferentes, uma ONG com 2.000 kg de capacidade recebia score de quase 100% mesmo estando longe. O motivo: o modelo só viu capacidades de até ~200 kg e distâncias de até 25 km no treino, e a regressão logística **extrapola sem limite** fora dessa faixa. A correção foi gravar a **faixa do treino** no modelo (`faixa_treino`) e, na inferência, levar cada valor ao limite dela, devolvendo a lista de `entradas_ajustadas`. Não houve novo treino: coeficientes e métricas são os mesmos. Hoje os scores variam de forma coerente (por exemplo, 55% para uma ONG a 6 km e 26% para uma a 84 km, com o mesmo lote), e a tela marca com * a distância acima de 25 km.

33. Como a IA roda dentro da API sem o scikit-learn?

O treino (`ml/train.py`) exporta os parâmetros (médias, escalas, vocabulário, IDF, coeficientes, faixa do treino) para `modelo_risco.json`. A API lê esse JSON e refaz a conta em Python puro (`ia/inferencia.py`). Um teste (`ml/tests/test_exportacao.py`) garante que o resultado do JSON é igual ao do pipeline scikit-learn. Vantagens: imagem menor, nenhuma execução de código ao carregar o modelo (o risco do `pickle`) e entradas validadas (texto até 200 caracteres, números finitos em faixas).

34. O enunciado cita Hugging Face Transformers. Usaram?

Não no modelo de risco: o componente textual é simples (nome do alimento) e o TF-IDF com regressão logística atende, sendo rápido e explicável. Os dados e o problema não justificavam um Transformer. A validação de alimento usa modelos de linguagem hospedados (Groq e Gemini) apenas como verificador do nome do alimento.

6. Segurança e STRIDE

Ameaças, controles e riscos que sobram.

35. O que é a matriz STRIDE e como foi usada?

É uma lista de ameaças organizadas em seis categorias (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). Cada linha descreve a ameaça, probabilidade e impacto, o **controle implementado** e o teste que o comprova. Hoje são **28 ameaças**, atualizadas a cada funcionalidade nova (oferta pública, portal da ONG, pedido de acesso, acompanhamento por código, migração automática, confirmação de recebimento). Controles só aparecem como "implementados" quando há teste ou execução registrada.

36. Como funciona o login e quem pode o quê?

- **Contas individuais** de gestor e de pessoas das ONGs: e-mail e senha. A senha é guardada só como **hash PBKDF2-SHA256** (600 mil iterações, sal por senha); a sessão (12 h) é um token aleatório de que o banco guarda apenas o hash. Há política de senha (10 a 128 caracteres, sem ser só números, comum, nem conter o e-mail ou o começo do nome).
- **Senha provisória:** quando o gestor cria ou redefine uma conta, o sistema gera uma senha que aparece **uma vez**; ela só permite trocar a senha. Desativar a conta, redefinir ou trocar a senha derruba as sessões na hora.
- **Perfis no servidor:** conta de ONG recebe 403 em qualquer rota do gestor, e no portal a ONG vem da conta, nunca de um id enviado pelo navegador. O gestor cria, desativa e redefine acessos na página **Acessos**.
- **Contra tentativas:** 5 falhas por e-mail em 15 min bloqueiam (inclusive e-mails que não existem, para não revelar quem tem conta) e 10 falhas por IP por minuto; a resposta e o tempo são iguais para e-mail inexistente, senha errada e conta desativada.
- **Link da ONG:** vira *convite*: com ele a pessoa cria a própria conta; depois o gestor pode **desativar** o link. O painel mostra a idade do link.
- **Não existe senha única de painel.** O `API_TOKEN` só vale para o Swagger e para scripts (aparece na auditoria como "Token da API") e é o caminho de recuperação: o primeiro gestor se cria com `python criar_gestor.py` ou pela API com esse token.
- **Doador:** sem conta; envia a oferta e a acompanha pelo código aleatório.

36a. Como é a auditoria?

Toda ação que altera dados e deu certo, e todo login (bom ou ruim), grava **quem** (conta, token da API ou link da ONG), **o quê**, em qual registro e **quando**, na tabela `auditoria_acoes`; o gestor vê na página Auditoria, com frases como "Aprovou a oferta #12". Não se guarda o corpo das requisições, senhas nem o e-mail digitado num login recusado, a gravação acontece antes de a resposta chegar e uma falha nela nunca derruba a operação. Isso responde ao risco de repúdio (R01 da matriz STRIDE).

37. Como o sistema se protege de abuso e negação de serviço?

- Servidor com threads, timeout de 10 s por conexão e no máximo 64 conexões simultâneas.
- 300 requisições por minuto por IP e 10 falhas de autenticação por minuto por IP (resposta 429).
- Rotas públicas: 5 ofertas por IP por hora (100 no total), 3 pedidos de acesso por IP por hora (30 no total), 20 códigos de acompanhamento inválidos por IP em 10 min, corpo de até 8 KiB, só JSON, **campo-armadilha** invisível para robôs e teto de pendentes.
- A validação barata roda antes de qualquer consulta à IA externa, para não gastar a cota com lixo.

> **Limitação:** os limites ficam na memória de cada instância e não substituem um proxy/WAF.

38. Como evitam injeção de SQL e XSS?

SQL: todas as consultas são parametrizadas; o Bandit bloqueia no pipeline qualquer montagem de SQL por texto (ele já apontou casos em código novo durante o desenvolvimento, e foram corrigidos antes do commit). **XSS:** o frontend escapa todo texto vindo da API e a **CSP** proíbe scripts e estilos inline, com um teste automatizado que varre os arquivos do frontend; também há `X-Frame-Options`, `nosniff`, `Referrer-Policy` e `Cache-Control: no-store`.

39. Dados pessoais podem vazar pela recomendação, pelo acompanhamento ou pelos indicadores?

Não por esses caminhos: a recomendação não inclui e-mail nem telefone; o acompanhamento mostra só etapa, alimento, quantidade e o nome da ONG que aceitou; os indicadores são agregados por região; e o contato do doador só aparece à ONG **depois que ela aceita**. As listagens completas exigem o token do gestor.

40. A migração automática ao iniciar não é arriscada?

É um risco consciente (T09 da matriz STRIDE) com controles: as migrações são **só aditivas e idempotentes** e um teste reprova qualquer `DROP`, `TRUNCATE`, `DELETE` ou `UPDATE`; cada arquivo roda na própria transação; se falhar, o servidor sobe e registra o erro; `AUTO_MIGRAR=0` desliga. Foi testada em banco vazio e em um banco que já tinha dados (nada foi perdido). O ganho é eliminar a falha mais provável de um deploy: esquecer a migração.

41. Quais riscos ainda existem?

Assumimos: (1) o token da API não tem identidade individual (só vale para scripts e recuperação, e aparece como tal na auditoria); (2) **não há MFA nem recuperação de senha por e-mail**: o gestor redefine e entrega uma senha provisória; (3) o token de sessão fica em `sessionStorage` (mitigado pela CSP, mas exposto se houvesse XSS); (4) o bloqueio por tentativas e os limites de taxa estão em memória por instância; (5) o link da ONG, enquanto ativo, não tem senha (o painel mostra a idade e permite desativar ou renovar); (6) a confirmação de recebimento ainda não tem comprovante. Esses pontos devem ser tratados antes de usar dados reais em grande escala.

42. O que a LGPD exige e o que foi feito?

Foi publicado o **aviso de privacidade** em `/privacidade`, ligado aos formulários com aceite obrigatório: quais dados, para quê, com quem se compartilha, por quanto tempo, quais os direitos e como solicitar. No desenho: minimização (só o nome do alimento vai à IA; a oferta não vira cadastro antes da aprovação; **recusar uma oferta ou descartar um pedido apaga os dados pessoais**), contato do doador restrito, código de acompanhamento só como hash, dados de demonstração claramente marcados e segredos fora do Git. Pendências: identidade por usuário e procedimento formal de retenção e exclusão.

7. DevSecOps, Docker e CI/CD

Padronização, automação e segurança contínua.

43. Como é o Dockerfile?

Imagem `python:3.12-alpine3.24`, dependências com **versão exata**, execução por **usuário sem privilégios** (UID 10001) e `HEALTHCHECK` em `/health`. O `.dockerignore` deixa de fora `.env`, `.git`, `.venv` e caches, e nenhum segredo é embutido na imagem: tudo vem do ambiente. A base Debian anterior foi trocada por Alpine porque o Trivy apontou 44 ocorrências HIGH (8 CVEs) sem correção.

44. E o Docker Compose?

Sobe a aplicação lendo o `.env` e aplica endurecimento: sistema de arquivos `read_only`, `tmpfs` em `/tmp`, `cap_drop: ALL`, `no-new-privileges` e porta publicada só em `127.0.0.1`. O banco é externo (Supabase), por isso o Compose não provisiona banco; as tabelas são criadas pelo próprio servidor ao iniciar.

45. O que o pipeline do GitHub Actions faz?

Roda em push na `main`, em pull requests e manualmente. Há três jobs:

- **python-security:** valida a sintaxe, roda os testes, **Bandit** (bloqueia severidade média/alta com confiança média/alta), **pip-audit** (bloqueia dependência com CVE conhecida) e confere o hash do Swagger UI vendorizado.
- **secret-scan: Gitleaks** sobre todo o histórico do Git.
- **container-security:** valida o Compose, constrói a imagem, sobe o container e testa saúde, frontend, OpenAPI, 401 sem token e CSP; **Trivy** analisa o Dockerfile/Compose e a imagem (vulnerabilidades, segredos e configuração; bloqueia HIGH/CRITICAL).

Relatórios são guardados como artefatos mesmo quando a análise reprova. As Actions estão **fixadas por SHA** e o workflow tem permissão mínima (`contents: read`).

46. O que é SAST e o que o Bandit e o Trivy cobrem?

SAST é a análise estática do código, sem executá-lo. O **Bandit** faz isso no Python (backend e `ml/`). O **Trivy** complementa olhando a imagem e as dependências (CVEs), segredos esquecidos e erros de configuração do Dockerfile/Compose. Na verificação final local: Bandit com zero achados (nem baixos), pip-audit sem vulnerabilidades, Gitleaks sem vazamentos no histórico e Trivy sem HIGH/CRITICAL, na imagem e na configuração.

> **Ressalva honesta:** "sem achados" vale para a cobertura e a base de vulnerabilidades daquele momento; o scanner de pacotes Alpine não cobre CVEs ainda sem correção. Não afirmamos ausência total de vulnerabilidades.

47. O que são as "vulnerabilidades mitigadas" do dashboard?

Achados que foram **corrigidos e confirmados por nova análise**: 44 ocorrências HIGH (8 CVEs) da base Debian, removidas ao trocar para Alpine (Trivy), e 2 achados baixos do Bandit em `ml/`. Total mostrado: 46, com 0 críticas ou altas em aberto. A fonte e as datas estão em `documentacao/05-devsecops-e-deploy.md`.

48. Os Security Gates do dashboard são ao vivo?

Não, e dizemos isso. São um **retrato local datado**, mas não digitado à mão: o script `ferramentas/gerar_seguranca.py` roda as ferramentas reais (Bandit, pip-audit, Gitleaks, Trivy e os testes) e grava o resultado, com as versões das ferramentas e o commit-base, em `ia/seguranca.json`; o que não puder ser executado aparece como "não executado", nunca como "sem achados". A execução oficial e contínua é a do GitHub Actions. Ligar o dashboard ao Actions exigiria um token de leitura do GitHub (a consulta sem credenciais não enxerga o repositório), o que fica como melhoria.

49. Há deploy contínuo?

O deploy é **manual de propósito** (`autoDeploy: false` no `render.yaml`): só se publica depois que o pipeline fica verde. Isso evita colocar no ar um commit que reprovou nos gates. A automação de publicação a partir do Actions é um passo futuro.

8. Deploy e operação

Render, Supabase e o dia da defesa.

50. Como a aplicação foi publicada?

No **Render** (plano gratuito) como Web Service Docker, descrito em `render.yaml`, em **https://alimenta-ia.onrender.com**. O banco é o **Supabase** (Session pooler, conexão por IPv4 com SSL). Os segredos (`DATABASE_URL`, `API_TOKEN`, `ADMIN_PASSWORD`, chaves da IA, `ONG_LINK_SECRET`) ficam no painel do Render, nunca no Git. Atrás do proxy do Render usa-se `TRUST_PROXY=1` para enxergar o IP real do cliente.

51. O que fazer após atualizar o código?

(1) Esperar o pipeline ficar verde; (2) fazer o **deploy manual** no Render; (3) conferir no log do Render a linha "Banco atualizado" (as migrações rodam sozinhas ao iniciar); (4) abrir `/health`, `/admin` e as páginas públicas. Se por algum motivo as migrações não rodarem, o sistema continua funcionando em modo reduzido (pergunta 15) e `python migrar.py` resolve.

52. E se a internet ou o Render falharem na defesa?

O plano gratuito do Render **suspende por inatividade**: o primeiro acesso pode levar cerca de 1 minuto, então abra a URL uns 2 minutos antes. Plano B: rodar localmente com `docker compose up` (ou `python server.py`) e popular um banco local com `python demo_seed.py`, que também imprime códigos de acompanhamento de exemplo em vários estados; ter capturas de tela guardadas.

9. Dashboard e indicadores

O que mostrar e como cada número é calculado.

53. Quais indicadores o dashboard mostra?

A aba **Indicadores** (atualiza sozinha a cada 15 s) mostra: **kg de alimentos salvos** (com a parcela confirmada pela ONG), **refeições geradas**, acurácia da IA, **tempo médio de match**, vulnerabilidades mitigadas e lotes com validade crítica; refeições entregues por **região atendida**; um **mapa do Brasil** com as regiões onde as ONGs receberam alimento (e um detalhe ampliado quando elas estão juntas); o **mapa de calor** região do doador × status; as métricas da IA; entregas e tempo de resposta; o status de segurança (SAST) e a tabela de **Security Gates**.

54. Como cada número é calculado?

Indicador	Cálculo
Kg salvos	Soma de `quantidade_kg` das doações com status **concluída**; cada doação conta uma vez
Refeições	Kg salvos ÷ 0,5 kg por refeição (suposição da equipe: apresentar como estimativa)
Região atendida	A região da **ONG que recebeu** (onde a carência foi atendida), não a do doador; doação concluída sem ONG vinculada aparece à parte
Tempo médio de match	Média entre o **envio da oferta** (ou o cadastro pelo gestor) e a criação do match, em horas
Confirmadas pela ONG	Entregas concluídas com origem "ong" ÷ concluídas
Validade crítica	Doações disponíveis que vencem em até 3 dias (regra temporal, não previsão)
Acurácia e recall	Métricas do conjunto de teste do modelo (dados simulados)

55. Por que "refeições por região" usa a região da ONG?

Porque o enunciado fala em destacar "áreas de maior carência atendida": o impacto social acontece onde o alimento **chega**, não onde foi doado. Por isso as refeições e o mapa agrupam pela região da ONG que recebeu; a visão por região do doador continua no mapa de calor, como "oferta". Antes, os dois usavam a região do doador, o que contava o impacto no lugar errado.

56. Como o mapa sabe onde colocar cada região?

Pela média das coordenadas das ONGs da região (quando cadastradas) ou, na falta, pelo centro da cidade citada no nome da região. O contorno do Brasil é **simplificado** (serve de contexto, não de medida) e desenhado em SVG, sem biblioteca nem serviço externo, o que mantém a CSP estrita. Cada círculo tem dica com kg, refeições e a origem da posição. Não há "mapa de carência" real porque não temos dados de demanda: isso está dito no painel e na documentação.

57. "Kg salvos" é confiável?

Mais do que antes, mas ainda é uma **medida sem comprovante**: conta o que foi concluído, e a maior parte vem da confirmação da própria ONG, que o painel mostra em percentual. Não há foto nem assinatura da entrega. Não se deve chamar de "salvo" o volume apenas disponível.

58. Os números da demonstração são reais?

Não, e isso deve ser dito. O script `demo_seed.py` cria dados **fictícios** (e-mails `@demo.alimenta.example`) para o dashboard ter o que mostrar: 48 doações, ofertas públicas em vários estados, matches, confirmações e ~2 t concluídas. Ele só roda em banco local (recusa a nuvem sem confirmação explícita) e `--limpar` remove apenas o que ele criou. Em apresentação, identifique-os como dados de demonstração.

10. Decisões, limites e perguntas difíceis

Para responder com segurança quando a banca pressionar.

59. O que o projeto ainda não faz?

- Rotas de coleta e tempo de deslocamento: a distância é em linha reta.
- Envio automático de e-mail ou push às ONGs: hoje o gestor usa o botão "Avisar ONG" (e-mail ou WhatsApp com a mensagem pronta) e as páginas se atualizam sozinhas por consulta periódica.
- MFA, recuperação de senha por e-mail e conta para o doador (ele só acompanha pelo código).
- Verificação automática do e-mail e do CNPJ (a conferência é do gestor) e comprovante da entrega.
- Dados reais de treino: o modelo usa dados simulados.
- Deploy automático, limites de taxa compartilhados entre instâncias e Security Gates ligados ao Actions.

60. Como sabemos que funciona?

Há **212 testes automatizados** (autenticação, limites, cabeçalhos, validações, fluxo de doação, ofertas e acompanhamento, portal da ONG, migrações, geografia, IA e indicadores), que rodam sem banco e no pipeline. Além disso, o fluxo completo foi exercitado de ponta a ponta em bancos locais descartáveis, pela API e pela interface (computador e celular): em banco vazio, em banco com dados e em banco **sem as migrações novas**. Os resultados estão em `documentacao/07-execucao-e-validacao.md`.

61. O que testar e corrigir mudou o projeto? Dê exemplos.

- **Modelo extrapolando:** scores de ~100% para ONGs fora da faixa do treino (pergunta 32).
- **Doadores aleatórios:** a oferta pública criava um "doador" mesmo sem aprovação; hoje só a aprovação cria.
- **Mensagens de sucesso apagadas:** no portal da ONG, a atualização automática limpava o aviso logo depois de uma ação.
- **CSP no mapa:** os rótulos usavam estilo inline e o navegador os bloqueava; foi corrigido e virou um teste que varre o frontend.
- **Telas "escondidas" que apareciam:** uma regra de CSS fazia a barra de sessão da ONG e um botão aparecerem mesmo marcados como ocultos; achado em captura de tela e corrigido com uma regra global.
- **SQL por texto:** o Bandit apontou duas consultas montadas por texto em código novo; foram trocadas por consultas parametrizadas antes do commit.
- **Impacto no lugar errado:** as refeições por região usavam a região do doador; agora usam a da ONG.

62. O que é o "impacto social mensurável" aqui?

Kg salvos e refeições estimadas por região atendida, tempo entre enviar uma oferta e ter uma ONG indicada, entregas confirmadas por quem recebeu e lotes de validade crítica atendidos. Cada um tem fórmula, fonte e limitação descritas. O objetivo do desenho é reduzir o tempo entre "tenho sobra" e "uma ONG está avisada": é o que a IA e o fluxo aceleram, e o doador acompanha o caminho do começo ao fim.

63. Como a IA evita ser injusta com ONGs de regiões menores?

Hoje o score considera capacidade, urgência e distância, e a decisão final é **sempre humana**. Como a faixa do treino limita o efeito de capacidade e distância, uma ONG grande ou muito próxima não domina o ranking sozinha. A avaliação de equidade (garantir que regiões ou ONGs menores não recebam sistematicamente menos oportunidades) está registrada como pendência e depende de dados reais para ser medida.

64. Se tivessem mais um mês, o que fariam primeiro?

Pela ordem de impacto: (1) treinar com dados reais de doação e descarte; (2) rotas e tempo de deslocamento; (3) MFA e recuperação de senha por e-mail; (4) e-mail automático para ONGs, verificação de e-mail e CNPJ e comprovante de entrega; (5) deploy contínuo e Security Gates ligados ao Actions.

65. Qual foi a maior decisão de engenharia e o maior aprendizado?

Tratar a segurança e a privacidade como parte do desenho, não como etapa final: validar tudo no servidor, negar por padrão, minimizar o que entra e o que sai (a oferta não vira cadastro antes da aprovação, contatos só após o aceite, só o nome do alimento à IA) e provar os controles com testes e com o pipeline. Aprendizado: testar de verdade (banco local, navegador, bancos sem migração) achou erros que os testes unitários não pegavam, e a transparência sobre limites (dados simulados, "sem achados" não é "sem vulnerabilidades") torna o projeto mais crível, não menos.

Rede Alimenta IA · Projeto Integrador FIAP 2026 · Davi Oliveira da Silva (RM 569108) · João Pedro Morangoni (RM 570073) · João Vitor Xavier de Carvalho (RM 570633). Material de estudo; consulte a pasta `documentacao/` para o detalhamento e as evidências.